

Marmara University, 2020

Modeling and Discrete Simulation

Subject 3

General Principles and Concepts DES

Mujdat Soyturk, Ph.D.
Associate Professor

Contents



- Concepts in Discrete-Event Simulation
  - The Event Scheduling / Time Advance Algorithm
  - World Views
- List Processing

Slides are derived from the course textbook "Banks, Carson, Nelson, Nicol. Discrete-Event System Simulation, 5th Edition, Prentice Hall, 2010" and from textbook's course slides.

Purpose

- Develops a common framework for the modeling of complex systems.
- Covers the basic blocks for all discrete-event simulation models.
- Introduces and explains the fundamental concepts and methodologies underlying all discrete-event simulation packages.
  - These concepts and methodologies are not tied to any particular simulation package.

Discrete-Event Simulation (DES)

- In DES, a system is modeled in terms of
  - its state at each point in time.
  - the entities that pass through the system
  - the entities that represent system resources
  - the activities and events that cause the system state to change.
- Deals exclusively with dynamic, stochastic systems.
- Discrete-event models are appropriate for those systems for which changes in system state occur only at discrete points in time.
  - This lecture covers general principles and concepts:
    - Event scheduling/time advance algorithm.
    - The three prevalent world views.
  - Introduces some of the notions of list processing.

Concepts in Discrete-Event Simulation

- System: a collection of entities that interact together over time, e.g., people and machines.
- Model: an abstract representation of a system.
- System state: a collection of variables that contains all the info necessary to describe the system at any time.
- Entity: any object or component in the system, e.g., a server, a customer, a machine.
- Attributes: the properties of a given entity.
- Lists: a collection of associated entities, ordered in some logical fashion, a.k.a, sets, queues and chains, e.g. customers waiting in the line in FIFO.
- Event: an instantaneous occurrence that changes the state of a system, e.g., an arrival of a new customer.

Concepts in Discrete-Event Simulation

- Event Notice: A record of an event to occur at the current or some future time along with some necessary data.
- Event list: a list of event notices for future events, ordered by time of occurrence, a.k.a. the future event list (FEL)
- Activity: a duration of time of specified length which is known when it begins, e.g., a service time.
- Delay: A duration of time unspecified indefinite length, which is not known until it ends.
- Clock: a variable representing simulated time.

Note: different simulation packages use different terminology for the same or similar concepts.

Concepts in Discrete-Event Simulation

- An **activity** typically represents a service time, an interarrival time, or any processing time whose duration has been characterized/defined by the modeler.
- An *activity's duration* may be specified:
  - Deterministic, e.g. always exactly 5 minutes.
  - Statistical,
  - A function depending on system variables and/or entity attributes.
- Duration of an activity is precomputed before it begins, so it is not affected by the occurrence of other events.
- When the activity starts, an event notice is created including the completion time.
  - For example: If the current simulated time is CLOCK = 100 minutes, and an inspection time of exactly 5 minutes is just beginning, then an event notice is created that specified the type of event and the event time (100+5 = 105 min).

Concepts in Discrete-Event Simulation

- A *delay's duration* is not specified by the modeler ahead of time but is determined by system conditions.
  - It ends when some conditions or one or more other events occur.
  - For example, a customer's delay in a waiting line may be dependent on the number and duration of service of other customers ahead in line and, whether a server has a failure during the delay.
  - also called a conditional wait.
- The systems we want to model are dynamic, that is changing over time:
  - System state, entity attributes, the number of active entities, the contents of sets, and the activities and delays currently in progress are all function of time and are constantly changing over time.

Concepts in Discrete-Event Simulation

Example: Able-Baker Call Center System. A discrete-event model has the following components:

- System state:
  - The number of callers waiting to be served at time *t*
  - Indicator that Able is idle or busy at time *t*
  - Indicator that Baker is idle or busy at time *t*
- Entities: neither the caller nor the servers need to be explicitly represented, except in terms of the state variables, unless certain caller averages are desired.
- Events:
  - Arrival
  - Service completion by Able
  - Service completion by Baker
- Activities:
  - Interarrival time.
  - Service time by Able
  - Service time by Baker
- Delay: a caller's wait in queue until Able or Baker becomes free

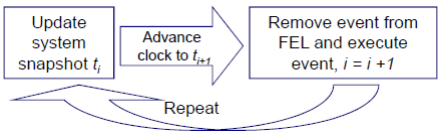
Concepts in Discrete-Event Simulation

- The definition of the model components provides a static description of the model.
- A description of the dynamic relationships and interactions between the components is also needed.
  - e.g., how does each event affect system state? || How activities are defined (i.e. deterministic, probabilistic)? || What events mark the beginning or end of each activity? || What is the system state at time 0?
- A discrete-event simulation is:
  - The modeling over time of a system all of whose state changes occur at discrete points in time.
  - Proceeds by producing a sequence of system snapshots that represent the evolution of the system through time.

Event Scheduling/Time Advance Algorithm

- The mechanism for advancing simulation time and guaranteeing that all events occur in correct chronological order is based on the ***Future Event List (FEL)***.
- At any given time *t*, the ***future event list (FEL)*** contains all previously scheduled future events and their associated event times ($t_1, t_2, \ldots$) (*event notices*)
  - FEL is **ordered** by event time, and the event time satisfy:

    $t \le t_1 \le t_2 \le t_3 \le \ldots \le t_n$ where t is the value of CLOCK



Prototype System Snapshot

Clock	System state	...	Future event list	...
t	(5,1,6)		$(3,t_1)$ – Type 3 event to occur at time t_1 $(1,t_2)$ – Type 1 event to occur at time t_2 $(1,t_3)$ – Type 1 event to occur at time t_3 . . . $(2,t_n)$ – Type 2 event to occur at time t_n	

At each imminent event time:

Step 1. Remove the event notice from the imminent event.

Step 2. Advance clock to imminent event time.

Step 3. Execute imminent event. Update system state.

Step 4. Generate future event list.

Step 5. Update cumulative statistics and counters.

Event-Scheduling/Time-Advance Algorithm performs these action within simulator.

List Processing in Event Scheduling

- Length and contents of the FEL are changing constantly as the simulation progress. Efficient management of the FEL has major impact in on the model's runtime.
- The management of a list.
  - The major list processing operations performed on a FEL are:
    - Removal of the imminent event
    - Addition of a new event to the list
    - Occasionally removal of some event (cancellation of an event).
  - Efficiency of search within the list depends on the logical organization of the list and how the search is conducted.
- When an event with event time t* is generated, its correct position on the FEL can be found via:
  - A top-down search, or
  - A bottom-up search

Future Events in Event Scheduling

- The system snapshot at ***time 0*** is defined by the initial conditions and the generation of the exogenous events.
- Then the first arrival at ***time 0*** (generation of first event) is scheduled on the FEL (inserted into the list).
- Arrival event (1st way of event generation) :
  - An exogenous event is a happening "outside the system" that impinges on the system.
  - For example, an arrival to a queuing system, at time 0, the 1st arrival event is generated and is scheduled on the FEL. When the clock eventually is advanced to the time of this first arrival, a second arrival event is generated. Actually, interarrival time is generated.
  - The end of an interarrival interval is an example of a primary event (primary event is managed by placing an event notice on the FEL.)
- This method of generating external arrival stream is called ***bootstrapping***.

Future Events in Event Scheduling

- Service completion event (2nd way of event generation):
  - When a customer completes the service at time t, if the next customer is present, then new service time is generated for the new customer.
  - Next service completion event is scheduled in FEL.
    - Because it is primary event. However, beginning service is a conditional event (it is not scheduled in the FEL).
    - Beginning service is triggered only on the condition that a customer is present and a server is free.
- A service time is an example of an activity.

Future Events in Event Scheduling

- 3rd way of event generation:
  - runtimes and downtimes for a machine subject to breakdowns.
    - At time 0, the first runtime will be generated → an end-of-runtime event is scheduled in the FEL.
    - When the end-of-runtime event occurs, a downtime is generated → an end-of-downtime is scheduled in the FEL.
    - When the end-of-downtime event occurs, a runtime is generated → an end-of-runtime is scheduled in the FEL.
    - These events continuously follow each other during the simulation.

Stopping Event

- Every simulation must have a stopping event, *E*.
  - It defines how long a simulation will run.
  - Two ways to stop a simulation
    - At time 0, schedule a stop simulation event at a specified future time T_E.
    - Run length T_E is determined by the simulation itself. Generally, T_E is the time of occurrence of some specified event E.

World Views (1)

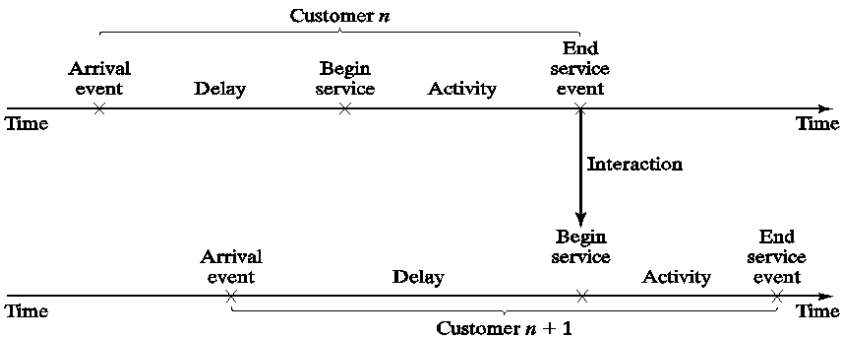
- The most prevalent world views are:
  - Event-scheduling world view (variable time advance.)
  - Process-interaction world view (variable time advance.)
  - Activity-scanning world view (fixed time increment.)
- Event-scheduling approach
  - Concentrates on events and their effect on system state
  - The one we discussed and summarized in the previous slides.

World Views (2)

- Process-interaction approach
  - Concentrates on the processes: a process is a time-sequenced list of events, activities and delays that define the life cycle of one entity as it moves through a system.
  - Usually, many processes are active simultaneously in a model, and the interaction among processes could be quite complex.
  - A popular approach because it has intuitive appeal and ease of simulation package implementation. But, process interactions are hidden from the modeler's view.
    - modeler has to have basic understanding of the concepts and detailed understanding of the built-in rules of the operation.
  - An example of a "customer process":

World Views (2)

- Process-interaction approach (2)
  - An example of a "customer process":



World Views (3)

Activity-scanning approach:

- Concentrates on activities of a model and those conditions that allow an activity to begin. (fixed-time incremental and rule-based approach to decide on an activity beginning)
- At each clock advance, the conditions for each activity are checked, and if the conditions are true, then the corresponding activities begin.
- Simple in concept but slow runtime on computers.

World Views (3)

Activity-scanning approach (modified version):

- The modified approach (combines event scheduling with activity scanning) is called the three-phase approach to overcome drawbacks.
- Events are considered to be activities of duration zero time units.
- Activities are divided into 2 categories:
  - B-type: activities that are bound to occur, e.g. primary events. Only B-Type events exist in the FEL.
  - C-type: activities or events that are conditional upon certain conditions being true. After the execution of B-type events, scanning the conditions for C-type events starts.
- Simulation proceeds with repeated execution of the 3 phases: from removal of events, then execute B-type events, and then execute C type events.

World Views (3)

Activity-scanning approach (modified version):

- The simulation proceeds with repeated execution of the 3-phase until it is completed:
  - **Phase A**. Remove the imminent event from the FEL and advance the clock to its event time.
  - **Phase B**. Execute all B-type events, that were removed from FEL.
  - **Phase C**. Scan the conditions that trigger each C-type activity.
- The 3-phase approach improves the efficiency of activity-scanning approach.
- It is good at handling complex resource problems.
- It guarantees that all resources being freed to be used at an instant time to reallocate to new tasks.

Lists: Basic Properties and Operations

- Lists are a set of ordered or ranked records.
  - In simulation, each record represents one entity or one event notice.
  - They have top or head (1st item) and bottom or tail.
  - Ways to traverse the list (to find the 2nd, 3rd, etc. items on the list) are necessary.
- An entity identifier and its attributes are fields in the entity record.
- Each record on a list has a field that holds a “next pointer” that points to the next record on the list.

Lists: Basic Properties and Operations

- The main operations on a list are:
  - Removing a record from the top of the list
  - Removing a record from any location on the list
  - Adding an entity record to the top or bottom of the list
  - Adding a record at an arbitrary position in the list, specified by the ranking rule.
- In the event-scheduling approach, when time is advanced and the imminent event is due to be executed:
  - First, the removal operation takes place.
  - If an arbitrary event is being canceled, or an entity is removed from a list based on some of its attributes to being an activity, then the second removal operation is performed
  - If a queue has the ranking rule *earliest* due date first, then, upon arrival at the queue, an entity must be added to the list determined by the due-date ranking rule.

Lists: Basic Properties and Operations

- For simulation on a computer:
  - All records are stored in arrays: arrays hold successive records in contiguous locations in computer memory, referenced by array index.
  - All entities and event notices are represented by structure (as in C) or classes (as in Java) allocated from RAM memory as needed, and tracked by pointers to a record or structure.